E2 — Arquitectura modular, Clean Code y SOLID

Explicación de la práctica y de los conceptos teóricos para repasar de cara al examen.

1. Qué se ha hecho en la práctica

Objetivo: convertir el script de la E1, que era un único fichero que se leía de arriba abajo, en un **paquete de Python** dividido en módulos, cada uno con una responsabilidad. El programa se ejecuta desde la línea de comandos con argumentos.

Estructura creada dentro de `1_programacion_avanzada_python/`:

```
ventas_app/
  __init__.py     # convierte la carpeta en paquete
  loader.py       # carga: Protocol + repositorios CSV/JSON + DataLoadError
  validator.py    # validación de filas + ValidationError
  metrics.py      # agregaciones y resumen de calidad
  cli.py          # punto de entrada: argparse + orquestación
  DESIGN.md       # principios SOLID y Clean Code aplicados
```

Ejecución (desde `1_programacion_avanzada_python/`):

```
python -m ventas_app.cli --input Datos/ventas.csv --output Datos/ventas_limpias.csv
```

Flujo del programa (lo que hace `cli.py`):

Paso	Módulo	Qué ocurre
1. Leer argumentos	`cli.py`	`argparse` obtiene las rutas de `--input` y `--output`
2. Cargar	`loader.py`	`load(path)` elige el repositorio (CSV o JSON) según la extensión y devuelve un DataFrame, o lanza `DataLoadError`
3. Validar	`validator.py`	`validar_ventas()` separa válidas e inválidas o lanza `ValidationError`
4. Métricas	`metrics.py`	importe por región, top 3 productos, clientes recurrentes
5. Exportar	`cli.py`	guarda `ventas_limpias.csv` y `calidad_datos.json`

Resultados: son idénticos a los de la E1 (150 filas, 140 válidas, 10 inválidas e importe total de 96 598,17). Esto confirma que la reorganización **no ha cambiado el comportamiento**, que es justo lo que se busca al refactorizar.

Qué se ha añadido respecto a la E1:

- Excepciones propias con mensajes claros.
- Comprobación de que existen las columnas obligatorias.
- `logging` en lugar de `print`.
- Constantes con nombre en lugar de números sueltos.
- Patrón Repository con `Protocol` (`SalesRepository`) y las clases `CsvSalesRepository` y `JsonSalesRepository` (extensión del enunciado).

---

2. Conceptos teóricos

2.1 Módulos y paquetes

- **Módulo:** cualquier fichero `.py`. Se importa por su nombre sin la extensión (`import loader`).
- **Paquete:** una carpeta que agrupa módulos. El fichero `__init__.py` le indica a Python que esa carpeta es un paquete. Puede estar vacío o contener solo una descripción.
- **Importar desde un paquete:**

```
from ventas_app import loader, metrics, validator
loader.load("Datos/ventas.csv")   # se llama a la función a través del módulo
```

- **Ventaja de dividir en módulos:** cada fichero es corto y se entiende solo; los cambios quedan localizados, y las piezas se pueden probar y reutilizar por separado.

2.2 `python -m` y `if __name__ == "__main__"`

- `python -m ventas_app.cli` ejecuta el módulo `cli` **como parte del paquete**. Así funcionan los imports del tipo `from ventas_app import ...`.
- Hay que lanzarlo desde la carpeta que **contiene** `ventas_app/`, porque Python busca el paquete en la carpeta actual.
- `__name__` es una variable que Python crea en cada módulo:
  - Vale `"__main__"` si el fichero se está ejecutando directamente.
  - Vale el nombre del módulo (`"ventas_app.cli"`) si se importa desde otro fichero.

```
if __name__ == "__main__":
    main()
```

Esto hace que `main()` solo se ejecute al lanzar el programa, y no al importarlo (por ejemplo, desde los tests).

2.3 Excepciones propias

```
class DataLoadError(Exception):
    """Error que se lanza cuando no se pueden cargar los datos."""
```

- Una excepción propia es una **clase que hereda de** `Exception`. No necesita código dentro, basta con el docstring.
- **Lanzar** un error: `raise DataLoadError("No existe el fichero: ...")`.
- **Capturar** un error:

```
try:
    ventas = loader.load(args.input)
except (loader.DataLoadError, validator.ValidationError) as error:
    logger.error(error)
    sys.exit(1)
```

- **Por qué crear errores propios:** el mensaje explica el problema en términos del negocio ("falta la columna X") en vez de una traza larga de pandas. Además, quien llama a la función puede capturar solo esos errores concretos.
- `sys.exit(1)` termina el programa con **código de salida 1**, que indica que ha habido un error. El código `0` significa que todo ha ido bien. Scripts y herramientas de CI usan este código para saber si el programa ha fallado.

2.4 `argparse`: argumentos por línea de comandos

```
parser = argparse.ArgumentParser(description="Limpieza y métricas de ventas")
parser.add_argument("--input", required=True, help="Ruta del fichero de ventas")
parser.add_argument("--output", required=True, help="Ruta del CSV de ventas limpias")
args = parser.parse_args()
args.input   # valor escrito después de --input
```

- Permite ejecutar el mismo programa con distintos ficheros **sin tocar el código**.
- `required=True` hace que el argumento sea obligatorio; si falta, argparse muestra un error.
- `help=` es el texto que aparece con `python -m ventas_app.cli --help`, que se genera automáticamente.

2.5 `logging` en lugar de `print`

```
logging.basicConfig(level=logging.INFO, format="%(levelname)s - %(message)s")
logger = logging.getLogger(__name__)
logger.info("Filas cargadas: %d", len(ventas))
logger.error("No existe el fichero")
```

Nivel	Uso
`DEBUG`	Detalles para depurar
`INFO`	Funcionamiento normal
`WARNING`	Algo raro, pero el programa sigue
`ERROR`	Algo ha fallado
`CRITICAL`	Fallo grave

- **Ventajas frente a `print`:** cada mensaje indica su nivel, se puede filtrar (por ejemplo, mostrar solo los errores), se puede enviar a un fichero y el formato se configura en un solo sitio.
- `level=logging.INFO` muestra los mensajes de nivel INFO o superior.
- `%s` y `%d` son marcadores que logging rellena con los valores que van después (`%s` para texto, `%d` para enteros).

2.6 Nuevas herramientas de `pathlib`

Uso	Resultado para `Path("Datos/ventas.csv")`
`ruta.exists()`	`True` / `False` según exista el fichero
`ruta.suffix`	`".csv"`
`ruta.with_name("calidad_datos.json")`	`Datos/calidad_datos.json` (misma carpeta, otro nombre)

2.7 SOLID

Son cinco principios de diseño que hacen que el código sea fácil de mantener y ampliar.

Letra	Principio	Idea en una frase
S	Single Responsibility (Responsabilidad única)	Cada módulo o clase tiene **un solo motivo para cambiar**
O	Open/Closed (Abierto/cerrado)	Abierto a **extender**, cerrado a **modificar** lo que ya funciona
L	Liskov Substitution (Sustitución de Liskov)	Cualquier implementación de una interfaz (o clase hija) debe poder **sustituir** a otra sin romper a quien la usa
I	Interface Segregation (Segregación de interfaces)	Mejor varias interfaces pequeñas que una grande con métodos que no todos usan
D	Dependency Inversion (Inversión de dependencias)	Depender de **abstracciones**, no de implementaciones concretas

Los cuatro aplicados en la práctica (el enunciado pide tres; ISP se añade porque se cumple de forma natural con el `Protocol`):

- **SRP:** `loader` solo lee, `validator` solo valida, `metrics` solo calcula y `cli` solo orquesta. Por ejemplo, si cambian las reglas de validación, solo se toca `validator.py`.

- **OCP:** en `validator.py`, la lista `COLUMNAS_POSITIVAS` se recorre con un bucle. Para exigir que otra columna sea positiva basta con añadirla a la lista, sin modificar la función.
- **DIP:** la función `load()` trabaja con la interfaz `SalesRepository` y no con un formato concreto. Además, `validator.py` y `metrics.py` solo reciben un `DataFrame`, así que no saben de dónde vienen los datos.
- **ISP:** el `Protocol` `SalesRepository` solo exige `load()`; un repositorio de lectura no tiene que implementar métodos que no usa.

Los otros dos principios, con ejemplos de la teoría:

- **LSP:** si el código espera un `SalesRepository`, cualquier implementación debe cumplir el mismo contrato: `load()` **devuelve un DataFrame o lanza** `DataLoadError`. Un repositorio que devolviera `None` rompería Liskov, porque quien lo usa espera un DataFrame.
- **ISP:** una interfaz pequeña (`SalesRepository` solo exige `load()`) es mejor que una gigante (`load`, `save`, `sync_s3`, `train_model`...). Quien solo quiere leer no debería estar obligado a implementar métodos que no usa.

SOLID en una línea por principio (resumen de la teoría):

```
S  una razón para cambiar
O  extender sin reescribir el núcleo
L  sustituir sin sorpresas
I  interfaces pequeñas
D  depender de abstracciones
```

DIP con `Protocol`: cómo está implementado en `loader.py`

Durante la práctica se probaron dos versiones: una sencilla (un `if` según la extensión dentro de `load()`) y esta, con clases. Se eligió la de clases porque el **Proyecto I pide OOP de forma explícita** (el criterio "Diseño OOP / modularidad" vale un 25 %) y es la que plantea el enunciado para la extensión (`JsonSalesRepository`).

Un `Protocol` define una "interfaz": qué métodos debe tener un objeto, sin decir cómo se implementan. Cualquier clase que tenga esos métodos la cumple **sin necesidad de heredar** de ella. Esto se llama *duck typing*: "si camina como un pato y hace cuac, es un pato".

```
class SalesRepository(Protocol):          # la abstracción
    def load(self) -> pd.DataFrame: ...

class CsvSalesRepository:                 # implementación concreta 1
    def __init__(self, path): self.path = path
    def load(self) -> pd.DataFrame: return pd.read_csv(self.path)

class JsonSalesRepository:                # implementación concreta 2 (extensión)
    def __init__(self, path): self.path = path
    def load(self) -> pd.DataFrame: return pd.read_json(self.path)

REPOSITORIOS = {".csv": CsvSalesRepository, ".json": JsonSalesRepository}

def crear_repositorio(path) -> SalesRepository:
    ruta = Path(path)
    ...                                   # comprueba existencia y extensión
    return REPOSITORIOS[ruta.suffix](ruta)

def load(path) -> pd.DataFrame:
    repositorio = crear_repositorio(path) # no importa si es CSV o JSON
    return repositorio.load()             # solo se usa el método de la interfaz
```

Conceptos de clases que aparecen:

- `class Nombre:` define una clase, que es una "plantilla" para crear objetos.
- `__init__` es el constructor: se ejecuta al crear el objeto y guarda sus datos. Por ejemplo, `CsvSalesRepository(ruta)` crea un objeto y guarda la ruta en `self.path`.
- `self` es el propio objeto. `self.path` es un **atributo** (un dato del objeto).
- `def load(self)` es un **método**, es decir, una función que pertenece a la clase.
- El diccionario `REPOSITORIOS` guarda **clases** (no objetos), y `REPOSITORIOS[".csv"](ruta)` crea el objeto. A esta función que decide qué clase instanciar se le suele llamar *factoría*.

Por qué cumple DIP y OCP a la vez: para admitir un formato nuevo (por ejemplo Excel), basta con crear `ExcelSalesRepository` con su método `load()` y añadirlo a `REPOSITORIOS`. No hay que modificar `load()`, `validator.py`, `metrics.py` ni `cli.py`. La extensión JSON se añadió exactamente así. En la versión sencilla, en cambio, habría que añadir otro `if` dentro de `load()`: es más simple, pero modifica una función que ya existe.

2.8 Clean Code

Mala práctica	Cómo se ha corregido
Nombres crípticos (`df2` , `res` , `x`)	Nombres que describen qué contienen o hacen: `importe_por_region` , `clientes_recurrentes`
Números mágicos (`> 0` , `.head(3)`)	Constantes con nombre en MAYÚSCULAS: `VALOR_MINIMO` , `NUM_TOP_PRODUCTOS`
`print` de depuración sueltos	`logging` con niveles
Funciones largas que hacen de todo	Funciones cortas con una sola tarea
Errores genéricos	Excepciones propias con mensajes claros

- **Número mágico:** un valor literal en el código cuyo significado no es obvio. Si se le da nombre como constante, se entiende al leerlo y, si cambia, se cambia en un solo sitio.
- **Convención:** las constantes se escriben en MAYÚSCULAS al principio del módulo.

2.9 Refactorizar

- **Refactorizar** es cambiar la estructura del código **sin cambiar lo que hace**.
- Forma de comprobarlo: el resultado final tiene que ser el mismo antes y después del cambio. En esta práctica, los mismos 150/140/10 y el mismo importe total que en la E1.

2.10 Clean Code: más reglas de la teoría

Comentarios: el *porqué*, no el *qué*. Un comentario que repite lo que el código ya dice sobra; conviene mejorar el nombre. El comentario útil explica una decisión que no es evidente:

```
# Mal (repite el código):
# Sumamos unidades * precio
importe = unidades * precio

# Bien (explica una decisión):
# El CSV histórico trae IVA incluido; no volver a aplicarlo aquí.
neto = bruto / 1.21
```

Aplicado en la entrega: durante el estudio se comentó el código línea a línea; en la versión entregada solo quedan los docstrings y los comentarios que explican decisiones (por ejemplo, `# Los valores no numéricos pasan a NaN, y NaN > 0 es False`). Los comentarios del tipo `import json # Lectura de JSON` se han eliminado porque solo repiten el código.

Efectos laterales. Un efecto lateral es cualquier cosa que cambia "el mundo" fuera del valor devuelto: escribir en disco, llamar a una API, modificar un DataFrame global o hacer `print`. Reglas:

1. Las funciones de negocio (`validator`, `metrics`) **no** escriben ficheros ni llaman a APIs.
2. La entrada y salida (**I/O**) vive en los **bordes**: `loader` (lectura) y `cli` (escritura).
3. Si una función modifica un DataFrame, debe trabajar sobre `copy()` o decirlo en su nombre.

Funciones de un solo nivel de abstracción. La función que orquesta llama a otras con nombres claros (`load`, `validar_ventas`, `importe_por_region`...) y no mezcla detalles de lectura, validación y escritura en el mismo bloque. Eso es lo que hace `main()` en `cli.py`.

Errores explícitos, no valores mágicos. Es mejor `raise DataLoadError(...)` que `return None`: el fallo es visible, quien llama decide qué hacer y los tests pueden comprobarlo con `pytest.raises`.

2.11 OOP útil: cuándo función y cuándo clase

Usar...	Cuándo
Función	No hay estado ni variantes (por ejemplo, `importe_por_region`)
Clase	Hay configuración (ruta, API key, umbrales), **variantes intercambiables** (CSV/JSON, mock/OpenAI) o se quiere un contrato (`Protocol`)

`dataclass`, para objetos que solo guardan datos (demo del profesor):

```
from dataclasses import dataclass

@dataclass(frozen=True)
class SalesRecord:
    region: str
    product: str
    units: float
    unit_price: float

    @property
    def amount(self) -> float:
        return self.units * self.unit_price
```

- `@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__` a partir de los atributos.
- `frozen=True` hace el objeto **inmutable**: no se pueden cambiar sus valores después de crearlo.
- `@property` permite usar un método como si fuera un atributo: `record.amount`, sin paréntesis.

Dos estilos de trabajo: todo en `DataFrame` (lo que se ha hecho en la práctica: rápido con pandas) o convertir las filas en objetos `SalesRecord` (lo que hace el demo: contratos más claros, pero más código).

2.12 Patrones de diseño útiles

Patrón	Problema que resuelve	Ejemplo
Repository	El resto del código no debe saber si los datos vienen de CSV, JSON, SQL o una API	`CsvSalesRepository.load()`
Pipeline stages	Un `run()` gigante mezcla todo	Etapas `load → validate → metrics → export`, cada una testeable
Strategy	Cambiar de algoritmo o proveedor obliga a poner `if/else` por todo el código	`AiProvider` con `MockProvider` y `OpenAIProvider` intercambiables
Facade	El usuario no quiere conocer todas las piezas internas	El CLI: un solo comando que orquesta todo

Patrones que todavía no hacen falta: Abstract Factory, Visitor, Singleton global y microservicios.

2.13 Over-engineering: cuándo parar

Preguntas de freno antes de añadir un patrón:

1. ¿Hay **hoy** una segunda implementación real (JSON, mock, otra API)?
2. ¿El patrón reduce código duplicado o solo añade ficheros?
3. ¿Un compañero lo entendería en 5 minutos?
4. ¿Lo pide el ejercicio o me lo estoy inventando?

Si la respuesta a 1–3 es "no", hay que simplificar. En este caso **sí** hay una segunda implementación real (JSON en la Parte A, y el Proyecto I pide OOP), así que el patrón Repository está justificado.

2.14 Notebook frente a paquete

Pregunta	Notebook	Paquete
¿Cómo se ejecuta?	"Run All", frágil y dependiente del orden	`python -m ventas_app.cli ...`
¿Se puede testear una función?	A menudo no	Sí, por etapas
¿Dónde está la validación?	"Por ahí"	En `validator.py`
¿Cambiar de CSV a JSON?	Reescribir celdas	Solo `loader.py`

Regla de oro: el notebook es el laboratorio (para explorar); el paquete es el producto (lo que se entrega).

2.15 Anti-patrones de arquitectura

1. `utils.py` de 800 líneas, un cajón de sastre.
2. `except Exception: pass`, que silencia los fallos. Hay que capturar excepciones concretas.
3. Mezclar HTTP, pandas y prompts de IA en una sola función.
4. *God object*: una clase que lo hace todo (`load`, `validate`, `call_openai`, `send_email`...).
5. Dependencias circulares: `a.py` importa `b.py` y `b.py` importa `a.py`.
6. Refactorizar solo cosméticamente (renombrar) y llamarlo "arquitectura".

---

3. Comandos usados

```
cd 1_programacion_avanzada_python
mkdir ventas_app
touch ventas_app/__init__.py ventas_app/loader.py ventas_app/validator.py \
      ventas_app/metrics.py ventas_app/cli.py ventas_app/DESIGN.md
python -m ventas_app.cli --input Datos/ventas.csv --output Datos/ventas_limpias.csv
python -m ventas_app.cli --help     # ver la ayuda generada por argparse
```

- `mkdir` crea una carpeta; `touch` crea ficheros vacíos.
- Error típico: `AttributeError: module 'X' has no attribute 'Y'`. Significa que el módulo no contiene esa función, normalmente porque el fichero no se guardó o el nombre está mal escrito.

---

4. Preguntas típicas de examen (E2)

1. **¿Qué diferencia hay entre un módulo y un paquete?** Un módulo es un fichero `.py`; un paquete es una carpeta con varios módulos y un `__init__.py`.
2. **¿Para qué sirve `__init__.py`?** Para indicar a Python que la carpeta es un paquete importable.
3. **¿Para qué sirve `if __name__ == "__main__"`?** Para que el código solo se ejecute cuando se lanza el fichero directamente, y no cuando se importa.
4. **¿Cómo se crea una excepción propia?** Con una clase que hereda de `Exception`: `class DataLoadError(Exception): ...`.
5. **¿Qué diferencia hay entre `raise` y `try/except`?** `raise` lanza un error; `try/except` lo captura y decide qué hacer con él.
6. **¿Qué significa `sys.exit(1)`?** Que el programa termina indicando que ha habido un error. `0` indica éxito.
7. **¿Por qué usar `logging` en vez de `print`?** Porque tiene niveles, se puede filtrar y redirigir, y el formato se configura en un solo sitio.
8. **¿Qué ventaja tiene `argparse`?** Permite cambiar los datos de entrada y salida sin tocar el código, y genera la ayuda (`--help`) automáticamente.

9. **Explica SRP con un ejemplo de la práctica.** `validator.py` solo valida; si cambian las reglas de validación, no hay que tocar ni `loader` ni `metrics`.
10. **Explica OCP con un ejemplo.** Añadir una nueva columna a `COLUMNAS_POSITIVAS` amplía la validación sin modificar la función `validar_ventas`.
11. **Explica DIP con un ejemplo.** `load()` depende del `Protocol` `SalesRepository` y no de una clase concreta. `CsvSalesRepository` y `JsonSalesRepository` lo cumplen, así que se pueden intercambiar. Por eso añadir JSON no obligó a tocar `metrics.py`.
12. **¿Qué es un número mágico y cómo se corrige?** Un valor literal sin explicación; se corrige sustituyéndolo por una constante con nombre.
13. **¿Qué es refactorizar y cómo se comprueba que se ha hecho bien?** Cambiar la estructura sin cambiar el comportamiento; se comprueba obteniendo los mismos resultados que antes.
14. **¿Qué es un `Protocol`?** Una interfaz que define qué métodos debe tener un objeto. Cualquier clase con esos métodos la cumple sin heredar de ella (*duck typing*).
15. **¿Cómo añadirías un nuevo formato de datos (por ejemplo Excel)?** Creando `ExcelSalesRepository` con un método `load()` y registrándolo en `REPOSITORIOS`. No cambian `load()`, `validator.py`, `metrics.py` ni `cli.py`.
16. **¿Qué son `__init__` y `self`?** `__init__` es el constructor, que se ejecuta al crear el objeto; `self` es el propio objeto, y permite guardar y leer sus atributos (`self.path`).
17. **¿Qué es un efecto lateral y dónde deben estar?** Cualquier cambio fuera del valor devuelto (escribir ficheros, llamar a una API, imprimir). Deben estar en los bordes: `loader` y `cli`, no en `validator` ni `metrics`.
18. **¿Por qué `validator` no debería escribir el CSV?** Por SRP: su único motivo de cambio son las reglas de validación. Escribir el CSV es I/O y le corresponde al `cli`.
19. **¿Qué problema resuelve el patrón Repository?** Aísla de dónde vienen los datos: el resto del código pide `load()` y no sabe si viene de CSV, JSON o una API.
20. **¿Cuándo es over-engineering usar Strategy o Repository?** Cuando solo existe una implementación real y el patrón añade ficheros sin quitar ningún problema.
21. **¿Qué hace `@dataclass(frozen=True)`?** Genera automáticamente el constructor y otros métodos, y hace que el objeto sea inmutable.
22. **Pon un ejemplo de algo que rompa LSP.** Un repositorio cuyo `load()` devuelva `None` en lugar de un DataFrame, porque quien lo usa espera un DataFrame.